

Rethinking Warping for Optical Flow on Edge Accelerators

Xiao’ao Song and Chahat Deep Singh
University of Colorado, Boulder¹

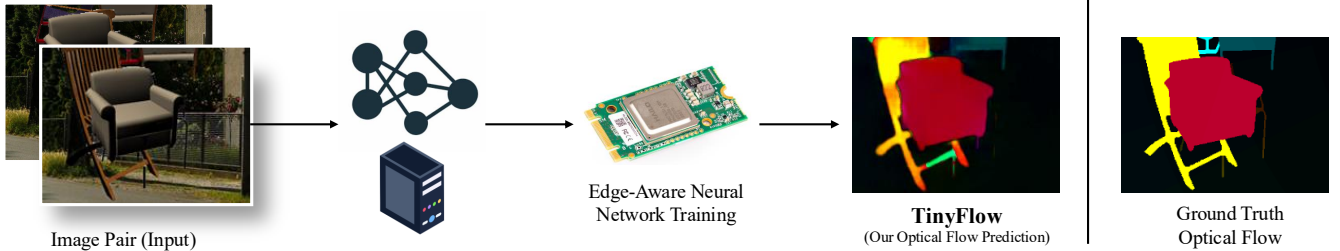


Fig. 1: **TinyFlow**. Our warp-free optical flow network with only 1.26M parameters utilizes a pair of input images to predict dense, accurate optical flow on edge AI accelerators.

Abstract—High-performance optical flow models are difficult to deploy on resource-constrained edge platforms. A major bottleneck is warping, an operation that shifts image features by the estimated motion to align the two frames. Edge AI accelerators often cannot execute it because of its dynamic nature i.e. memory accesses depend on the input. This paper presents **TinyFlow**, a tiny warp-free optical flow network for real-time edge deployment. **TinyFlow** keeps iterative refinement but replaces warping with a fixed-window correlation built from ordinary convolutions, so the entire network can be quantized, and compiled for edge AI accelerators. We also demonstrate that quantization-aware training and inference-time iteration scaling keep the deployed INT8 accuracy within 0.2 EPE of the Float32 model. **TinyFlow** outperforms existing edge optical flow baselines in both Float32 and INT8 accuracy while using 54% fewer parameters than the strongest baseline.

I. INTRODUCTION

Optical flow estimation aims to predict pixel-wise motion between consecutive image frames. It is a fundamental problem in computer vision and robotics because accurate motion estimation provides important cues for dynamic scene understanding. Optical flow has been widely used in robot navigation [1], visual odometry [2], structure-from-motion [3], object tracking [4], and dynamic scene understanding [5]. Recent learning-based optical flow methods have achieved significant improvements in accuracy. However, most high-performance methods are designed and evaluated primarily on desktop-grade GPUs. These models often require high computational resources, large memory capacity, making them difficult to deploy for onboard computing on resource-constrained robotic platforms.

The growing demand for autonomous robots, drones, and mobile systems has increased the need for perception algorithms that can run in real time on low-power edge

devices. Several recent works have explored lightweight vision models for such platforms. For example, Bouwmeester et al. [6] demonstrated real-time dense optical flow on a Bitcraze Crazyflie nano quadcopter using a GAP8 microprocessor. Singh et al. [7] proposed a lightweight depth-prediction network for palm-sized drones and achieved 4.5 Hz on a Raspberry Pi Compute Module 4. Raju et al. [8] further showed that optical flow can be accelerated to 100 Hz of inference speed at UIN8 precision on the Google Coral Edge TPU with very low power consumption at 1.08 W. In addition, Krispin-Avraham et al. [9] deployed a 3D object detection model on a low-power Hailo-8 AI accelerator achieved real-time inference at 5Hz approximately.

Despite this progress, deployable optical flow networks remain single feed-forward CNNs, and their accuracy trails modern iterative methods. The accuracy of those methods comes from refinement loops that repeatedly warp features with the current flow estimate, typically through `grid_sample`. Warping is precisely the operation that edge AI accelerators cannot execute, because its memory accesses depend on the data at runtime. The operations that make optical flow accurate are the same operations that prevent its deployment.

We propose **TinyFlow**, a tiny optical flow network that resolves this conflict. **TinyFlow** keeps the iterative refinement loop and removes the warp. It replaces warping with a fixed-window correlation whose offsets are constants, so the flow estimate selects among channel values rather than steering memory addresses. The entire network exports, quantizes, and compiles for edge AI accelerators such as the Hailo-8.

The main contributions of this paper are as follows:

- 1) We propose **TinyFlow**, a 1.26M-parameter warp-free optical flow network that retains iterative refinement and compiles to a single static INT8 graph.
- 2) We present a deployment study covering

¹Authors are with Perception, Robotics, AI, Sensing (PRAISE) Lab, University of Colorado, Boulder. Email: {xiaoao.song, chahat.singh}@colorado.edu.

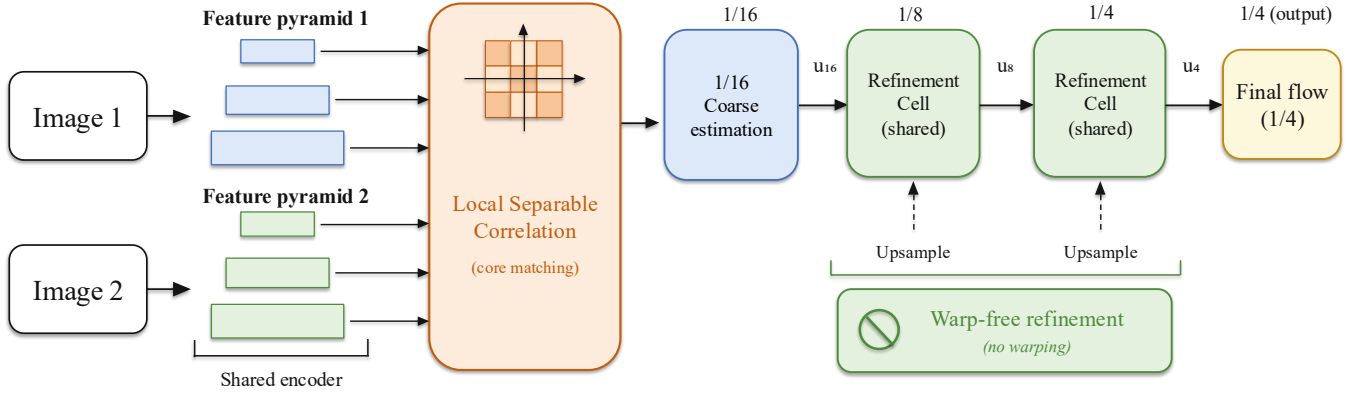


Fig. 2: TinyFlow Architecture

edge-compilable correlation, quantization-aware training, model souping, and inference-time iteration scaling.

- 3) We evaluate TinyFlow and demonstrate that it outperforms EdgeFlowNet on Sintel and FlyingChairs in both Float32 and INT8 while using 54% fewer parameters.

II. METHODOLOGY

A. Problem Formulation

Let $I_1, I_2 \in [0, 1]^{H \times W \times 3}$ be two consecutive RGB frames. The optical flow $\mathbf{u}$ assigns every pixel $\mathbf{x} = (x, y)$ of I_1 a displacement $\mathbf{u}(\mathbf{x}) \in \mathbb{R}^2$ such that $\mathbf{x} + \mathbf{u}(\mathbf{x})$ is the corresponding location in I_2 . We estimate optical flow in a coarse-to-fine manner at scales $s \in \{16, 8, 4\}$, where the estimate at scale s has spatial size $(H/s, W/s)$.

B. The Problem with Dynamic Resampling

Many optical flow networks refine flow by explicitly warping second-frame features with the current flow estimate: $\hat{f}_2^s(\mathbf{x}) \triangleq f_2^s(\mathbf{x} + \mathbf{u}^s(\mathbf{x}))$, where $f_1^s, f_2^s \in \mathbb{R}^{C_s \times H_s \times W_s}$ are learned features at scale s . The right-hand side is usually evaluated by bilinear resampling at flow-shifted, generally non-integer coordinates, often using operators such as `grid_sample`.

This operation is problematic for edge accelerators or NPUs [10] because it requires input-dependent memory access and spatial sampling primitives that are often unsupported by compilation toolchains, mainly due to dynamic sizes [6], [11]. If unsupported, warping falls back to the host CPU and forfeits the accelerator’s speed advantage. *Our goal is therefore to preserve coarse-to-fine flow estimation while producing an edge-compilable inference graph with no dynamic resampling in the forward pass.*

C. Multi-Scale Feature Representation

We now describe an architecture that meets this goal using fixed (rather than dynamic), edge-compilable operations only. A shared encoder extracts a feature pyramid $f_i^s = \phi_s(I_i)$ from each input image I_i , where $i \in \{1, 2\}$ indexes the frame and $s \in \{16, 8, 4\}$ the scale. The encoder is

implemented using depthwise-separable convolutions and pointwise nonlinearities to reduce parameter count and improve quantization friendliness.

Before computing correlation, we pass the features through a small learnable projector $\hat{\mathbf{f}} = \text{BN}(\text{Conv}_{1 \times 1}(\mathbf{f}))/\sqrt{C}$, where C is the number of channels. The projector keeps correlation values in a stable numeric range. Batch normalization bounds the feature magnitudes and the $1/\sqrt{C}$ factor cancels the growth of the inner product with channel count. The 1×1 convolution is initialized as an identity mapping, so training starts from plain correlation and learns a better projection only if one helps. This module is also quantization-friendly because a convolution, a batch normalization, and a fixed scale all fuse into a single INT8-legal operation. Correlation is computed after this projection unless stated otherwise.

D. Warp-Free Matching via Fixed-Window Correlation

With the projected feature pyramids in place, the remaining question is how to match them across frames without any flow-dependent resampling. Instead of computing a residual cost volume around warped features, we compute a fixed-window local correlation volume directly between the two feature maps:

$$C_0^s(\mathbf{x}, \delta) \triangleq \langle \hat{f}_1^s(\mathbf{x}), \hat{f}_2^s(\mathbf{x} + \delta) \rangle, \quad \delta \in \mathcal{D}(r_s), \quad (1)$$

where $\langle \cdot, \cdot \rangle$ denotes the channel-wise inner product and $\mathcal{D}(r_s) = \{-r_s, \dots, r_s\}^2$. The resulting cost volume has $K_s = |\mathcal{D}(r_s)| = (2r_s + 1)^2$ channels per spatial location.

Relationship to Warping: The standard warped residual cost volume can be written as

$$C_u^s(\mathbf{x}, \delta) \triangleq \langle \hat{f}_1^s(\mathbf{x}), \hat{f}_2^s(\mathbf{x} + \mathbf{u}^s(\mathbf{x}) + \delta) \rangle. \quad (2)$$

If $\mathbf{u}^s(\mathbf{x}) \in \mathbb{Z}^2$, this is simply a reindexing of the fixed cost volume,

$$C_u^s(\mathbf{x}, \delta) = C_0^s(\mathbf{x}, \delta + \mathbf{u}^s(\mathbf{x})). \quad (3)$$

Warping therefore creates no new matching evidence. It only recenters the search window around the current flow

estimate. Our design keeps the window fixed and instead chooses the radii $\{r_s\}$ large enough that the correct displacement stays inside it.

E. Coarse-to-Fine Warp-Free Flow Estimation

Flow estimation proceeds coarse to fine. At the coarsest scale, we concatenate the cost volume of Eq. 1 with the first-frame features, $\mathbf{x}^{16} = [C_0^{16}, f_1^{16}]$, and a small convolutional head h_{16} predicts the initial flow $\mathbf{u}^{16} = d_{16} \cdot \tanh(h_{16}(\mathbf{x}^{16})/d_{16})$, where the scaled hyperbolic tangent bounds the output to $\pm d_{16}$ pixels.

At each finer scale $s \in \{8, 4\}$, the previous estimate is upsampled to initialize $\mathbf{u}_0^s = 2 \cdot \text{Up}(\mathbf{u}^{2s})$, where $\text{Up}(\cdot)$ is bilinear upsampling and the factor of two preserves pixel units. A weight-tied update block g_s then refines the flow for $t = 0, \dots, T_s - 1$ steps, with $T_s=4$ during training. Each step predicts a bounded flow correction $\Delta \mathbf{u}_t^s$ and adds it to the current estimate, $\mathbf{u}_{t+1}^s = \mathbf{u}_t^s + \Delta \mathbf{u}_t^s$, with

$$\Delta \mathbf{u}_t^s = d_s \cdot \tanh(g_s([C_0^s, f_1^s, \mathbf{u}_t^s])/d_s). \quad (4)$$

The cost volume C_0^s is never recentered. The current flow enters the update block as an input feature instead. Each correction is bounded to $\pm d_s$ pixels in the same way as the initial flow. We set $d_{16}=4$, $d_8=2$, $d_4=1$. Bounded outputs keep flow regression numerically stable and representable in 8-bit encodings. The final prediction is $\mathbf{u}_{T_4}^4$ at 1/4 resolution, which is bilinearly upsampled to full resolution for evaluation.

F. Edge-Compilable Correlation

A naive implementation of Eq. 1 gathers a patch of $\hat{f}_2^s$ around every pixel, which exports as dynamic indexing that edge toolchains reject. The key observation is that every step of correlation can instead be written as a convolution with constant weights. A convolution with a one-hot kernel shifts a feature map by a fixed offset. We therefore shift $\hat{f}_2^s$ once for every offset in the window, using a depthwise convolution whose one-hot kernels are fixed at construction time. Multiplying each shifted copy with $\hat{f}_1^s$ and summing over channels with an all-one 1×1 convolution yields exactly the inner products of Eq. 1, stacked as the K_s channels of the cost volume C_0^s . No weight in this construction is learned and no memory address depends on the input. To the compiler, the cost volume is three ordinary layers.

G. Training Objective

Let $\mathbf{u}^*$ denote the ground-truth flow at full resolution. The ground truth at scale s is its downsampling with values divided by s , $\mathbf{u}^{*,s} = \frac{1}{s} \text{Down}_s(\mathbf{u}^*)$.

1) *Robust Supervision*: We penalize flow error with the Charbonnier function $\rho(\mathbf{z}) = (\|\mathbf{z}\|_2^2 + \epsilon^2)^\gamma$ with $\gamma \in (0, 1)$, which behaves like an L_1 loss on large errors and stays smooth near zero. The data loss at scale s averages this penalty over all pixels,

$$\mathcal{L}_s = \frac{1}{|\Omega_s|} \sum_{\mathbf{x} \in \Omega_s} \rho(\mathbf{u}^s(\mathbf{x}) - \mathbf{u}^{*,s}(\mathbf{x})), \quad (5)$$

where Ω_s is the pixel grid at scale s .

2) *Deep Supervision*: Every refinement iteration is supervised, not only the last one. With $\mathbf{u}_t^s$ the prediction after iteration t as in Eq. (4), the deep supervision loss is

$$\mathcal{L}_{\text{ds},s} = \frac{1}{T_s} \sum_{t=1}^{T_s} w_t \mathcal{L}_s(\mathbf{u}_t^s), \quad w_t \propto t, \quad (6)$$

so later iterations carry more weight.

3) *Edge-Aware Smoothness*: Let G_4 be the grayscale image I_1 downsampled to 1/4 resolution. We regularize the finest flow with $\mathcal{L}_{\text{smooth}} = \sum_{\mathbf{x}} \exp(-\eta \|\nabla G_4(\mathbf{x})\|_1) \|\nabla \mathbf{u}^4(\mathbf{x})\|_1$, which encourages smooth flow in homogeneous regions while allowing sharp changes at image edges. The total loss is

$$\mathcal{L} = \lambda_{16} \mathcal{L}_{16} + \lambda_8 \mathcal{L}_8 + \lambda_4 \mathcal{L}_4 + \mu_8 \mathcal{L}_{\text{ds},8} + \mu_4 \mathcal{L}_{\text{ds},4} + \nu \mathcal{L}_{\text{smooth}}. \quad (7)$$

H. Quantization-Aware Training

Training with the objective of Eq. 7 produces an accurate Float32 network, but the deployed network runs in INT8 precision on neural accelerators. Simply quantizing a trained network is not enough. Converting our checkpoints from Float32 to INT8 after training cost up to +1.0 end point error (EPE) on Sintel, and the loss grew the longer the network had been trained. The fix is to let the network experience 8-bit arithmetic *during* training. We insert simulated quantization at exactly the points where the accelerator’s compiler will quantize [12], so the weights learn to tolerate rounding. The quantized student also benefits from a better teacher than ground truth alone. A distillation loss [13] lets the accurate but non-deployable warp-based model guide it throughout training. Float and quantized accuracy disagree, so we select and average checkpoints by their *quantized* accuracy rather than their float accuracy. Training data is FlyingChairs and FlyingThings3D only. We evaluate on MPI-Sintel, which is never seen during training, fine-tuning, or quantization calibration. These measures collectively reduce the quantization penalty from +1.0 EPE to below +0.2 EPE. Sintel clean EPE only increases from 6.13 to 6.29 and final from 6.95 to 7.13.

III. RESULTS

A. Training Setup

We train the base network with a learning rate of 2×10^{-4} and run quantization-aware fine-tuning at 6×10^{-5} . All experiments use a desktop PC with an AMD Ryzen 7 9800X3D CPU, 32 GB of RAM, and an NVIDIA GeForce RTX 5080 GPU. We report EPE on Sintel Clean, Sintel Final, and the FlyingChairs validation split, together with parameter counts.

B. Training Mixture, Model Souping, and Inference

In order to improve the model accuracy, we perform three additional steps. (i) We mix FlyingThings3D [14] into the FlyingChairs [15] training data. This substantially reduces the EPE on large motions, where displacements exceed 40 pixels, from about 47 to 43. (ii) We then build a model soup [16] by averaging the weights of the three best fine-tuning epochs into a single checkpoint. The souped

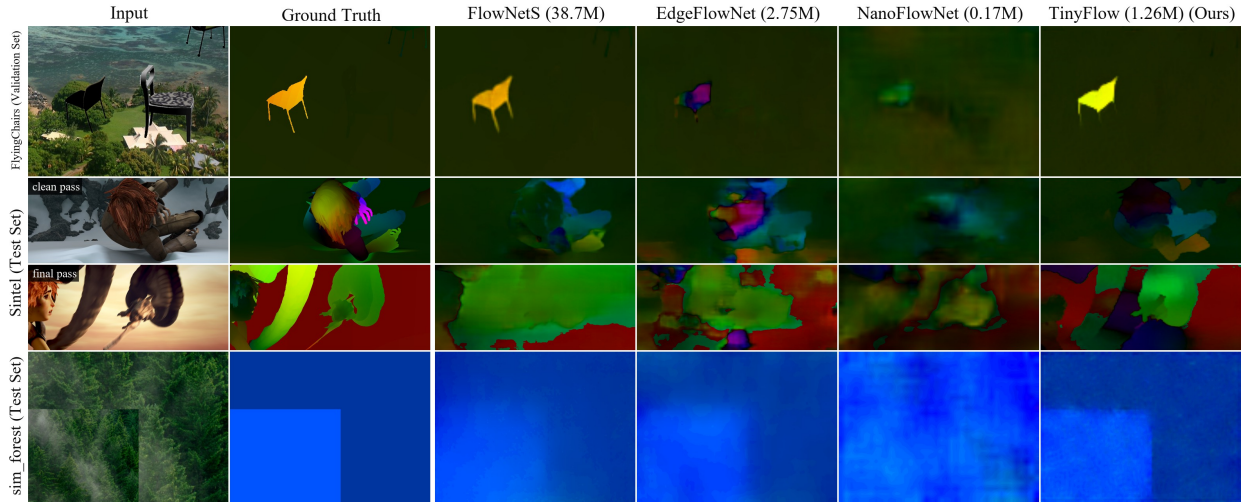


Fig. 3: **Qualitative comparison.** Image hue/color encodes optical flow direction and brightness encodes magnitude, normalized per row to the ground truth’s scale. First column shows the input data: FlyingChairs validation, Sintel clean and final passes, and a simulated forest flight. Second column depicts the optical flow ground truth. The remaining columns show the predictions of FlowNetS, EdgeFlowNet, NanoFlowNet, and TinyFlow (ours). TinyFlow recovers the cleanest and sharpest motion segments among the deployable models at the smallest parameter count. EdgeFlowNet fragments large motion and NanoFlowNet produces structured noise.

TABLE I: Trained on FlyingChairs and FlyingThings3D. Tested on MPI-Sintel and FlyingChairs validation set.

Model	Params	Prec.	EPE↓		
			Sintel Clean	Sintel Final	Chairs
FlowNetS (ref.)	38.7M	F32	4.86	6.38	1.85
EdgeFlowNet	2.75M	F32	6.17	7.30	3.59
		INT8	6.46	7.62	3.91
NanoFlowNet [†]	0.17M	F32	8.30	9.21	8.28
		INT8	8.61	9.16	8.17
TinyFlow (Ours)	1.26M	F32	6.13	6.95	2.66
		INT8	6.29	7.13	2.71

[†]All models are evaluated on 384×512 px images except NanoFlowNet, which is evaluated at its native resolution of 112×160 px. At 384×512 NanoFlowNet collapses to 20.7/22.5 EPE respectively.

model outperforms each of its ingredients. (iii) At inference, we increase the number of refinement iterations. The iterative update blocks at the 1/8 and 1/4 scales are weight-tied, so the same weights can simply be applied more times. Training uses four iterations at each scale and we deploy with eight, which reduces Sintel clean EPE by about 0.5 with no retraining. Accuracy saturates at eight iterations and degrades beyond.

C. Experimental Analysis

As shown in Table I, our model outperforms EdgeFlowNet across all F32 and INT8 evaluations while using 54% fewer parameters (1.26M vs. 2.75M). Under INT8, we reach 6.29/7.13 EPE on Sintel Clean/Final versus 6.46/7.62 for EdgeFlowNet, and on FlyingChairs our INT8 EPE is 31% lower (2.71 vs. 3.91). TinyFlow also substantially outperforms NanoFlowNet, which is evaluated at its most favorable setting, its native 112×160 resolution, since

at 384×512 it collapses. Although FlowNetS achieves higher accuracy, it uses about $30\times$ more parameters (38.7M). Overall, our model provides the best trade-off between accuracy and model size among all deployable models.

Fig. 3 further demonstrates the differences between the models. Our model produces coherent motion regions with sharp object boundaries across all three datasets. This is particularly evident in the simulated forest sequence, where our model clearly separates the relative motion of the two planes. In comparison, EdgeFlowNet frequently fragments large motion regions, while NanoFlowNet produces blur and structured noise that leave object boundaries and motion direction indistinct. FlowNetS also separates the two planes, but its motion boundaries are blurred.

IV. CONCLUSION

We presented TinyFlow, a 1.26M-parameter optical flow network that keeps iterative refinement while removing warping. Fixed-window correlation moves the data dependence of warping from memory addresses into channel values, so the full coarse-to-fine estimator compiles to a single static INT8 graph and outperforms EdgeFlowNet on every measured setting with 54% fewer parameters.

REFERENCES

- [1] H. Chao, Y. Gu, and M. Napolitano, “A survey of optical flow techniques for robotics navigation applications,” *Journal of Intelligent & Robotic Systems*, vol. 73, no. 1, pp. 361–372, 2014.
- [2] T. Pandey, D. Pena, J. Byrne, and D. Moloney, “Leveraging deep learning for visual odometry using optical flow,” *Sensors*, vol. 21, no. 4, p. 1313, 2021.
- [3] J. L. Schonberger and J.-M. Frahm, “Structure-from-motion revisited,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4104–4113.
- [4] S. Karpuzov, G. Petkov, S. Ilieva, A. Petkov, and S. Kalitzin, “Object tracking based on optical flow reconstruction of motion-group parameters,” *Information*, vol. 15, no. 6, p. 296, 2024.
- [5] C. Luo, “Motion learning for dynamic scene understanding,” Ph.D. dissertation, Johns Hopkins University, 2022.

- [6] R. J. Bouwmeester, F. Paredes-Vallés, and G. C. H. E. de Croon, "NanoFlowNet: Real-time dense optical flow on a nano quadcopter," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023.
- [7] C. D. Singh, B. He, C. Fermüller, C. Metzler, and Y. Aloimonos, "Minimal perception: enabling autonomy in resource-constrained robots," *Frontiers in Robotics and AI*, vol. 11, p. 1431826, 2024.
- [8] S. R. K. P. Raju, R. Singh, M. Velmurugan, and N. J. Sanket, "Edgeflownet: 100fps@ 1w dense optical flow for tiny mobile robots," *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 128–135, 2024.
- [9] I. Krispin-Avraham, R. Orfaig, and B.-Z. Bobrovsky, "Real-time 3d object detection using innovizone lidar and low-power hailo-8 ai accelerator," *arXiv preprint arXiv:2412.05594*, 2024.
- [10] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Ai and ml accelerator survey and trends," in *IEEE High Performance Extreme Computing Conference (HPEC)*, 2022.
- [11] S. R. K. Pinnama Raju, R. Singh, M. Velmurugan, and N. J. Sanket, "EdgeFlowNet: 100FPS@1W dense optical flow for tiny mobile robots," *IEEE Robotics and Automation Letters*, vol. 9, 2024.
- [12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [14] N. Mayer, E. Ilg, P. Häusser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox, "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4040–4048.
- [15] A. Dosovitskiy, P. Fischer, E. Ilg, P. Häusser, C. Hazirbas, V. Golkov, P. van der Smagt, D. Cremers, and T. Brox, "FlowNet: Learning optical flow with convolutional networks," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 2758–2766.
- [16] M. Wortsman, G. Ilharco, S. Y. Gadre *et al.*, "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in *ICML*, 2022.